

Dependency and physics-library review

Date: 2026-09-15. Scope: current repository plus current upstream documentation. Alternatives were researched, not installed or benchmarked. Companion: game audit (repository companion report).

Direct answer

Yes. The project implements the general physics solver as well as its bike/rider simulation in TypeScript. Its only runtime npm dependency is Three.js 0.186.0. Three.js supplies rendering, not bike physics.

“From scratch” describes the implementation accurately. “Purely from first principles” does not: it uses established rigid-body techniques, simplified tyre/rider models and explicit gameplay assistance, including external lean torque. This is normal territory for game physics; the relevant question is whether the result is predictable, learnable and maintainable.

A focused comparison project is warranted. Compare existing v2 with Rapier 2D and Planck.js. Do not approve a full rewrite based on a library feature list. No library automatically provides Trials handling, a rider controller or authored techniques.

1. Actual dependencies

`pnpm list --depth 0` returned these installed versions; some manifest development dependencies use ranges.

Purpose	Installed packages
Runtime rendering	three 0.186.0
Build/language	typescript 5.9.3, vite 7.3.6, tsx 4.23.13
Tests/harness	vitest 3.2.7, playwright 1.62.0, jsdom 26.1.0
Lint	eslint 9.39.5, @eslint/js 9.39.5, typescript-eslint 8.70.0, globals 16.5.0
Types	@types/three 0.186.0, @types/node 24.13.4, @types/jsdom 30.0.0

Vite's transitive esbuild is explicitly resolved by `vite.config.ts` for the inline loader. Media evidence scripts also invoke tools such as FFmpeg; those are development tools, not runtime physics dependencies. No Rapier, Planck, Box2D, Jolt, Cannon, Ammo or Matter package is declared. The physics modules import project-local code.

2. What was reimplemented

Component	Source	Responsibility
Default v2 world	<code>src/physics/v2/bike.ts</code> - 1,991 lines	Integration, contacts, constraints, suspension, faults, dynamic objects, snapshots
Collision geometry	<code>src/physics/collision.ts</code> - 517 lines	Custom contact geometry and queries
Deterministic math	<code>src/physics/dmath.ts</code> - 103 lines	Trigonometric functions with controlled cross-engine arithmetic
Drive and tyres	<code>src/physics/v2/engine.ts</code> , <code>tyre.ts</code>	Torque, braking, throttle response, slip/friction and assistance
Rider model	<code>src/physics/v2/rider.ts</code> and world solver	Pose targets, movement limits, rigid rider and servo forces
Legacy v1	<code>src/physics/bike.ts</code> - 2,390 lines	Retained former physics implementation

The default selected by `src/physics/index.ts:42` is v2. It advances at 120 Hz using semi-implicit Euler integration, sequential impulses, speculative contacts and position correction. Chassis mass is 58 kg; rear/front wheels are 8/7 kg; the rider is a separate 75 kg rigid body. A crash adds articulated ragdoll bodies.

Suspension has spring/damper forces, travel limits and bump stops. Rider target movement can produce a hop. Tyres, engine reaction and braking affect pitch. The rider applies paired forces, but `v2/bike.ts:1133` also adds external lean torque and damping. These assists mean the game is not only emergent motion from passive masses.

The present engine's strongest investment is replay/snapshot control. The fresh audit passed nine determinism checks on the development harness. Its weakest established area is human control/acceptance, not demonstrated raw physics CPU cost. Those are different problems.

3. The earlier developer did consider libraries

The historical design at `git show 7990f05:docs/design/physics.md` contains a custom-versus-Rapier-versus-Planck comparison, selecting custom and retaining Planck as a fallback. It would be inaccurate to say no alternatives were considered.

The current `docs/design/physics.md:1041` summarizes concerns about size, asynchronous WASM startup and tyre modelling. Some are legitimate constraints, but they should be tested again:

- Snapshot completeness, contact customization and reproducible arithmetic are real integration requirements.
- Old bundle/startup estimates were not independently benchmarked in this audit.
- Planck is pure JavaScript/TypeScript, so WASM startup is not an objection to Planck. [Planck upstream](#).
- A library need not supply a finished tyre/rider model to be useful: it can replace collision and constraint infrastructure while retaining custom game forces. Whether that interaction remains stable must be demonstrated.

4. Candidate assessment

These are suitability judgments from architecture and upstream capabilities, not measured rankings. The scene is rendered in 3D, but riding physics is planar; a 2D solver can drive Three.js transforms directly.

Candidate	Why relevant	Main uncertainty	Position
Rapier 2D	JS/WASM bindings, rigid bodies/joints, documented deterministic execution and world snapshots	Suspension/rider/tyre integration, WASM startup and snapshot throughput	First comparison candidate
Planck.js	Pure JS/TS, mature Box2D-style 2D constraints, dedicated motorized wheel/suspension joint	Bit-identical browser replay and full solver-state restore	Second comparison candidate
Rapier 3D	Same ecosystem with spatial bodies	Extra degrees of freedom for an essentially planar game	Consider if lateral physics becomes a requirement
JoltPhysics.js	3D engine bindings, joints/motors, actual motorcycle example	Distribution-specific determinism, state capture and planar constraints	Strong 3D contingency
cannon-es	JS/TS engine and vehicle helpers	Less direct fit for detailed wheel/ledge contacts and strict replay contract	Lower priority
Ammo.js	Bullet through Emscripten; extensive 3D functionality	Binding/build complexity and snapshot/replay integration	Lower priority
Matter.js	Accessible 2D browser engine	More custom suspension/motor work than Planck's dedicated wheel joint	Lower priority

Rapier 2D

Rapier's official JS documentation guarantees matching results across browsers/platforms for the same version, initial conditions and operation order. Its warning matters: application-side initialization must also be deterministic. It supports whole-world byte-array snapshots and restoration. These capabilities directly address this project's replay and beam-search needs. Its license is Apache-2.0. [Determinism](#), [serialization](#), [joints](#), [project/license](#).

The inference is that Rapier deserves a prototype, not that it already satisfies this game's performance or feel. Benchmark the chosen package/version's compressed bytes, startup and repeated restore cost. Verify that tyre impulses and rider control can be integrated without unstable or duplicated friction forces.

Planck.js

Planck's wheel joint supplies suspension translation, a rotational motor and a linear spring/damper. That is a direct structural match for this bike. It is MIT-licensed. [Upstream project](#), [wheel joint implementation](#).

However, its rotation helper uses platform transcendental functions, and the wheel-joint serializer does not include its cached solver impulses. Treat basic serialization as distinct from byte-identical continuation after an exploratory simulation branch. These observations identify tests to run; they do not demonstrate that Planck will diverge in every browser or scenario. [Rotation implementation](#), [wheel serialization source](#).

Modern native Box2D's determinism claims do not automatically transfer to Planck or arbitrary browser ports. Box2D itself distinguishes reproducible forward simulation from restoring all hidden running state. [Box2D determinism discussion](#).

3D and lower-priority options

JoltPhysics.js has a motorcycle example and multiple WASM/asm.js builds. Native Jolt documents an optional cross-platform deterministic configuration; the specific JS build's settings and state restoration still need verification. MIT license. A motorcycle demo is relevant precedent, but not a Trials rider implementation.

[Bindings](#), [demo list](#), [determinism constraints](#), [license](#).

Cannon's raycast vehicle samples ground with rays. For this project, actual wheel shapes contacting ledges and drums appear a more direct fit; that is an architectural inference, not a benchmark result. Matter's documented constraints focus on distance/spring behavior. Ammo supplies a broad Bullet port with more binding machinery than a planar bike necessarily needs. [Cannon vehicle API](#), [Matter constraints](#), [Ammo upstream](#), [Ammo license](#).

All candidates above are available as open-source projects. Check the exact pinned distribution's license, transitive notices and release activity during the prototype; none was installed here. Package popularity alone should not decide the solver.

5. Proposed comparison project

Suggested time box: 3-5 working days. Output: a measured selection decision, not an engine rewrite.

Preserve current v2 as the control. Build one minimal Rapier 2D bike and one Planck bike behind the existing input/track/state boundary.

Stage A - prove the integration constraints first

1. Represent chassis, circular wheels, suspension and a separately moving rider mass.
2. Adapt a small set of existing colliders and record the same quantized player input format.
3. Prove repeated replay equality and snapshot -> branch -> restore -> replay equality. Include contact transitions and crash state.
4. Test the actual selected JS/WASM version in headless Chromium and WebKit, followed by an actual iPhone Safari run.

PhysicsSnapshot currently requires { v: 1, f64: Float64Array, u8: Uint8Array } (src/core/types.ts:299). Rapier's byte snapshot needs an adapter or a versioned contract change; Planck must preserve solver internals rather than only recreate visible bodies. Include game/rider/controller state as well as the engine world. Keep old recording physics versions explicit.

Stage B - compare contact behavior and control

Use flat launch/braking, sustained wheelie, recoverable angled landing, steep plank, ledge, drum and bunny-hop scenes. Keep masses and target behavior comparable while documenting solver-specific tuning. Compare successful and unsuccessful techniques; include small input variations near crash boundaries. The current property tests exclude branches that fault, leaving those boundaries undermeasured.

Measure	Decision requirement
Determinism	Each engine repeats itself exactly for the same version/input; different engines need not produce identical trajectories
Snapshot throughput	Repeated beam-search branch/restore is correct and practical, including contacts and warm-start state
Runtime	Physics p95, total frame pacing and memory fit the game's existing budgets
Startup	Measure full compressed production delta and cold-start time, including WASM initialization
Restart	One-tick logical restart plus measured input-to-visible-control latency
Handling	Unfamiliar humans can hold/recover manoeuvres using real keyboard/touch input; track attempts and abandonment
Motion quality	Blind clips of the same manoeuvres, including suspension, rider reaction and failures
Maintenance	Less custom collision/constraint code without equally complex workaround code

Use small, equally sized unfamiliar-player groups as exploratory evidence; record the sample and all failed sessions. Avoid treating scripted adaptive macros as human play. An exploratory result can select a direction without pretending to prove population-wide preference.

Stage C - choose, retain or stop

Migrate only if a candidate preserves determinism/restart, demonstrates better control/contact behavior and materially reduces maintenance. Stop a candidate early if snapshot fidelity or key contacts cannot be made correct in the time box. If neither shows a meaningful gain, retain v2 and target its measured wheelie/recovery gaps.

The project can improve substantially without changing physics engines. Production boot, graphics performance, art/audio integration and evidence quality are independent work. A library experiment should answer whether solver ownership is helping this game, rather than becoming another prolonged tuning loop.